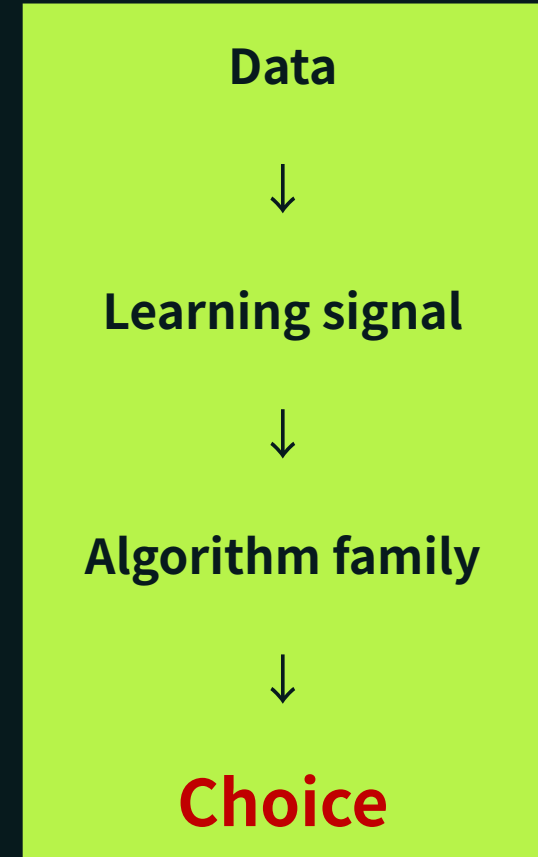


Machine Learning Algorithm Family Tree

See the family before the algorithm



Machine Learning Algorithm Family Tree

ML algorithms fall into four families

The full infographic — later slides zoom in on each family

Supervised Labels y — regression, classification

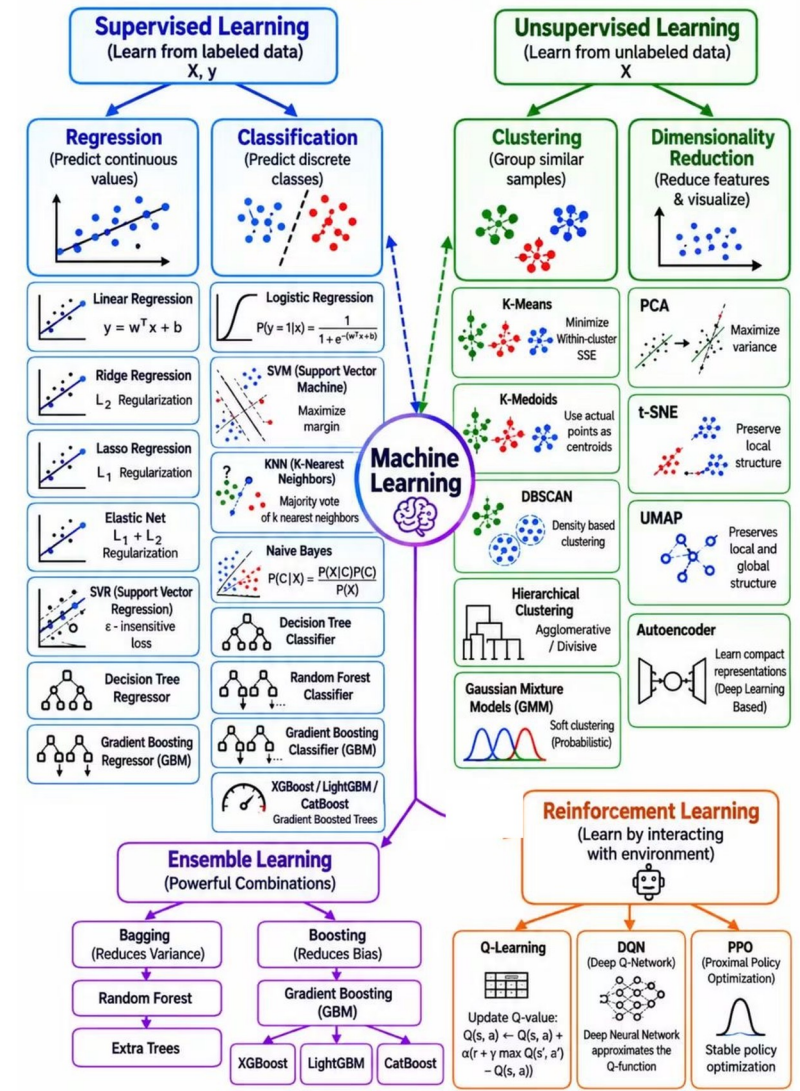
Unsupervised Only X — clustering, reduction

Ensemble Combine models — bagging, boosting

Reinforcement Act to maximise reward

Learning signal first, algorithm name second

Machine Learning Algorithm Family Tree



Each family optimises a different objective

One line each — what every family minimises or maximises

Supervised · minimise prediction error

$$\hat{f} = \arg \min_f \frac{1}{n} \sum_{i=1}^n L(y_i, f(x_i))$$

Unsupervised · learn the data structure

$$\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^n \log p_{\theta}(x_i)$$

Ensemble · weighted sum of weak models

$$F(x) = \sum_{m=1}^M \alpha_m h_m(x)$$

Reinforcement · maximise discounted reward

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\pi} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right]$$

An algorithm's objective tells you its family at once

01

Supervised Learning

Learn the mapping from inputs X to labels y

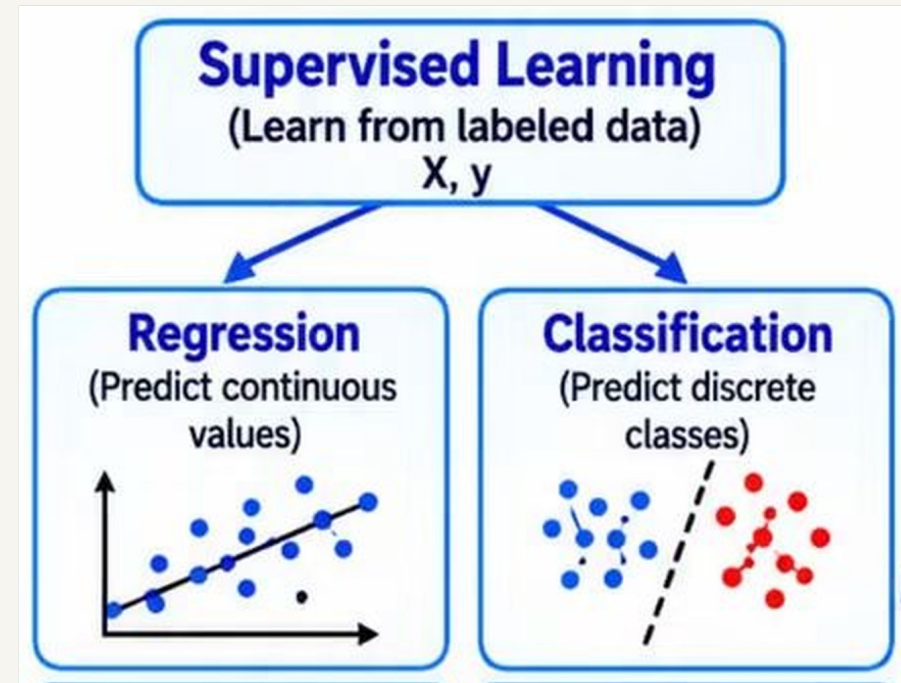
The type of target splits supervised learning in two

Learn from labeled data (X, y) — top panel of the original

REGRESSION vs CLASSIFICATION

Number or class?

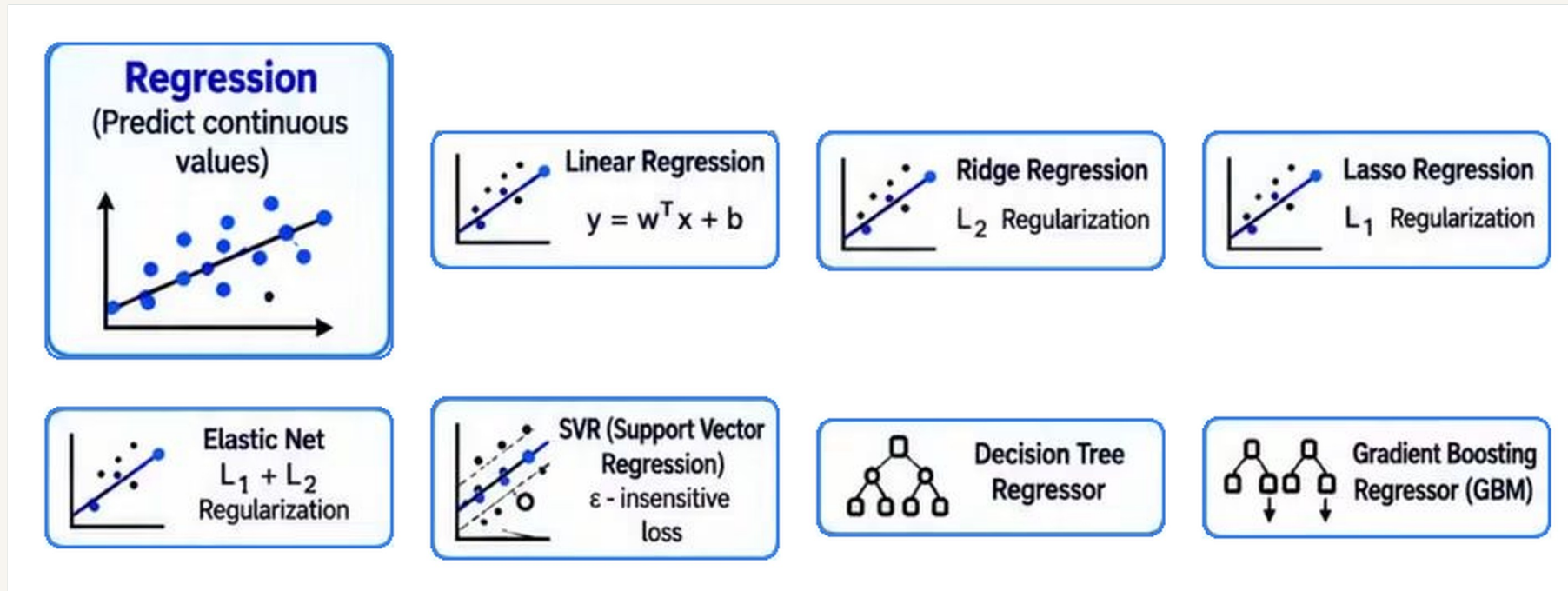
- Regression — price, return
- Classification — default
- Type of y sets the loss



Regression learns error size; classification learns a boundary

Regression runs from linear models to boosted trees

Regression (predict continuous values) — seven algorithms in the original



Linear → regularised → margin (SVR) → trees and boosting

The more flexible the model, the more overfitting matters

The penalty shapes the coefficients

Ridge (L2) shrinks all coefficients; Lasso (L1) sets some exactly to zero

Ridge · L2 penalty

$$\min_w \sum_i (y_i - w^\top x_i)^2 + \lambda \|w\|_2^2$$

Lasso · L1 penalty (selects variables)

$$\min_w \sum_i (y_i - w^\top x_i)^2 + \lambda \|w\|_1$$

Elastic Net · L1 + L2

$$\min_w \sum_i (y_i - w^\top x_i)^2 + \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2$$

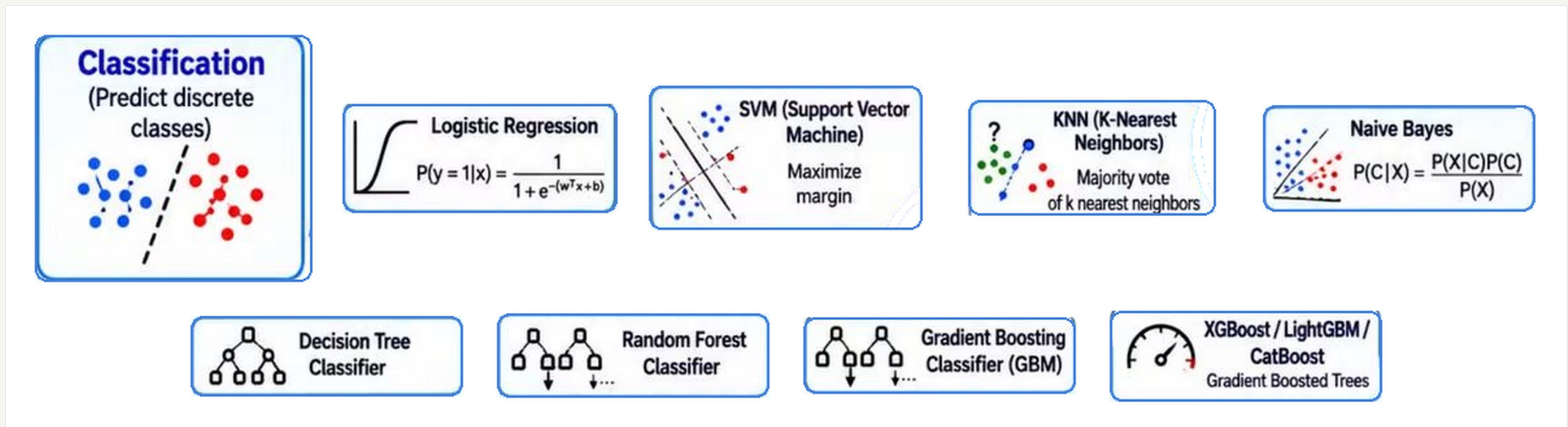
SVR · ϵ -insensitive loss

$$L_\epsilon(y, f(x)) = \max(0, |y - f(x)| - \epsilon)$$

λ trades bias for variance — choose it by cross-validation

Classifiers: probability, margin, neighbours, trees

Classification (predict discrete classes) — eight algorithms in the original



Logistic, NB: probability · SVM: margin · KNN: neighbours · trees: splits

On tabular data, GBMs (XGBoost, LightGBM, CatBoost) are the usual starting point

Each classifier draws its boundary differently

The sigmoid turns a linear score into a probability; KNN looks only at the k nearest

Logistic regression • minimise cross-entropy

$$\mathcal{L} = - \sum_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right]$$
$$p_i = \sigma(w^\top x_i + b)$$

SVM • maximise the margin ($2 / \|w\|$)

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad \text{s.t.} \quad y_i (w^\top x_i + b) \geq 1$$

KNN • majority vote of k neighbours

$$\hat{y}(x) = \arg \max_c \sum_{i \in N_k(x)} \mathbf{1}[y_i = c]$$

Naive Bayes • conditional independence

$$P(C \mid x) \propto P(C) \prod_{j=1}^d P(x_j \mid C)$$

Need probabilities: start with logistic. Many features, few samples: SVM

02

Unsupervised Learning

Find groups and low-dimensional structure from X alone

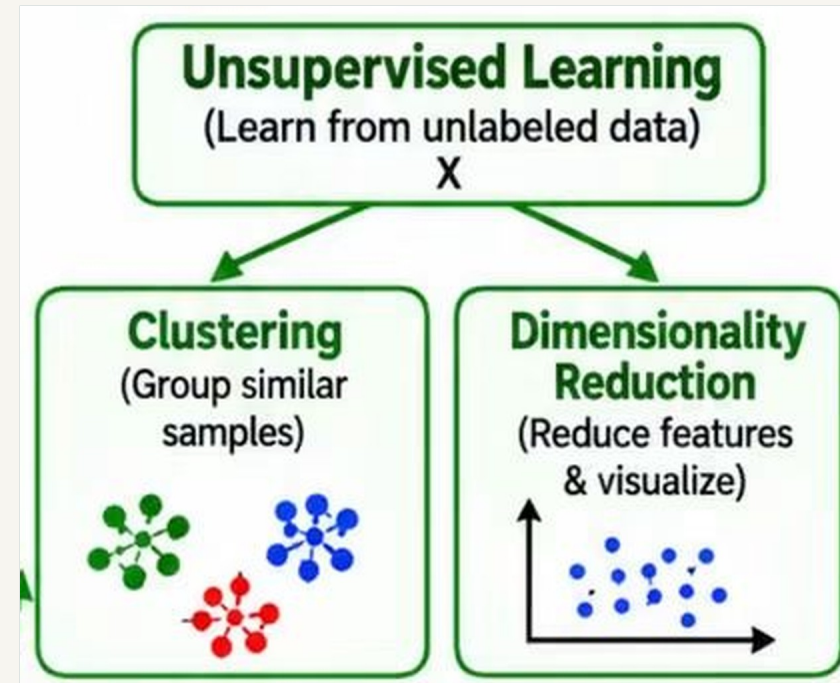
Unsupervised: clustering or dimension reduction

Learn from unlabeled data (X) — top panel of the original

CLUSTERING vs DIM. REDUCTION

Rows or columns?

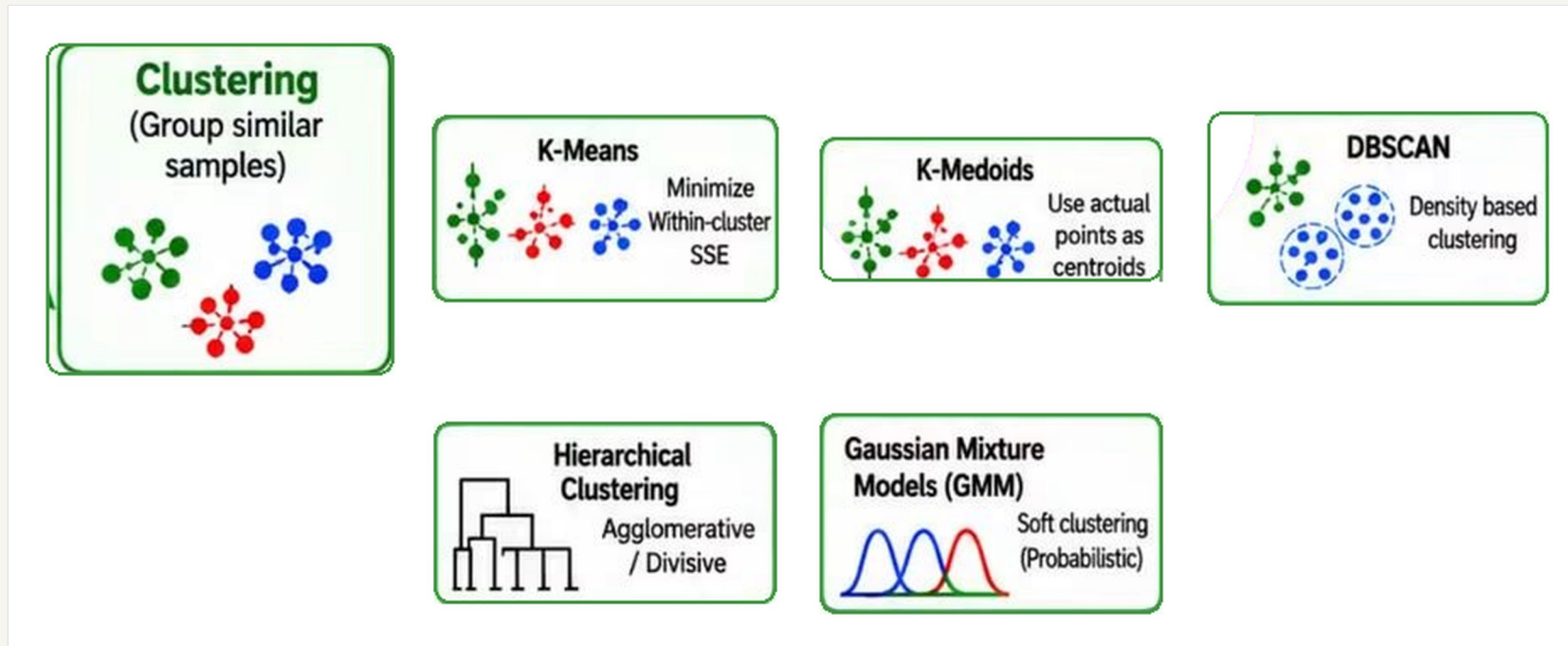
- Clustering — groups rows
- Reduction — fewer columns
- Also used as preprocessing



With no labels, checking the result matters as much as the model

Clustering by distance, density, hierarchy or probability

Clustering (group similar samples) — five algorithms in the original

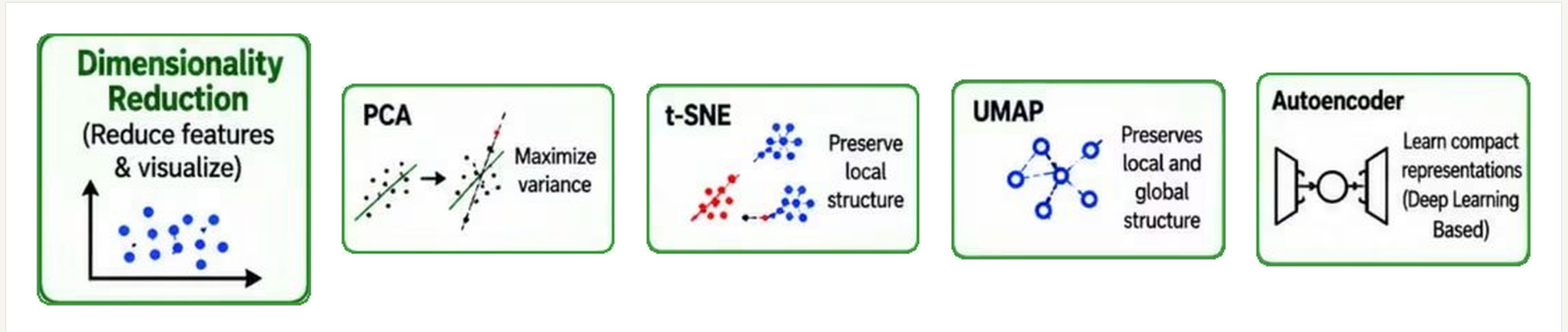


K-Means: distance · DBSCAN: density · hierarchical: merges · GMM: probability

Odd-shaped clusters or many outliers: try DBSCAN or GMM first

Reduction: from variance to nonlinear structure

Dimensionality reduction (reduce features, visualise) — four algorithms in the original



PCA: variance • t-SNE: local • UMAP: local + global • autoencoder: neural

t-SNE and UMAP are for plots — do not read their distances literally

Unsupervised algorithms have clear objectives too

Σ = covariance matrix · f_ϕ, g_θ = encoder and decoder

K-Means · minimise within-cluster SSE

$$\min_{\{C_k, \mu_k\}} \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2$$

GMM · Gaussian mixture (soft clustering)

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

PCA · max-variance direction = eigenvector

$$w_1 = \arg \max_{\|w\|=1} w^\top \Sigma w \Rightarrow \Sigma w_1 = \lambda_1 w_1$$

Autoencoder · minimise reconstruction error

$$\min_{\phi, \theta} \sum_i \|x_i - g_\theta(f_\phi(x_i))\|^2$$

K-Means is GMM with equal covariances and hard 0/1 assignments

03

Ensembles and Reinforcement Learning

Combining models, and learning a policy from rewards

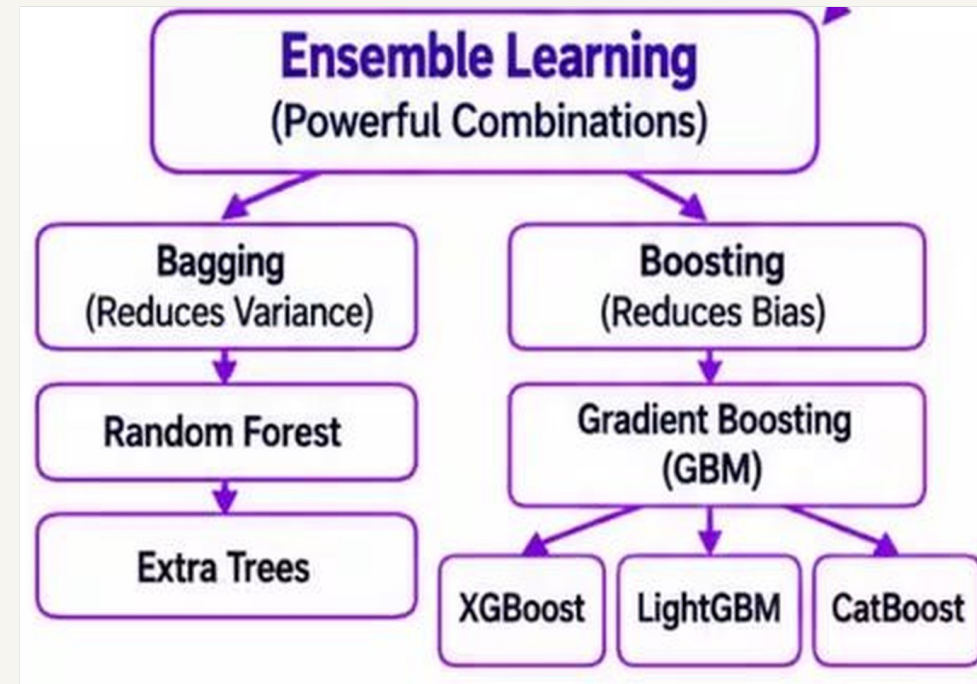
Bagging cuts variance; boosting cuts bias

Ensemble learning (powerful combinations) — lower-left panel of the original

BAGGING vs BOOSTING

Parallel or sequential?

- Bagging — average
- Boosting — fix residuals
- Pick by error source



Deep trees overfit: bag them. Shallow trees underfit: boost them

The two ensembles differ in their formulas

Correlation between models limits bagging; the learning rate sets boosting's step

Bagging · average of B models

$$\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^B \hat{f}_b(x)$$

Bagging · variance of the average

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

Boosting · additive model

$$F_m(x) = F_{m-1}(x) + \nu h_m(x)$$

Gradient boosting · fit the negative gradient

$$h_m \approx -\frac{\partial L(y, F(x))}{\partial F(x)} \bigg|_{F=F_{m-1}}$$

Lower correlation ρ , bigger variance cut — why random forests sample features

Boosting is gradient descent in function space

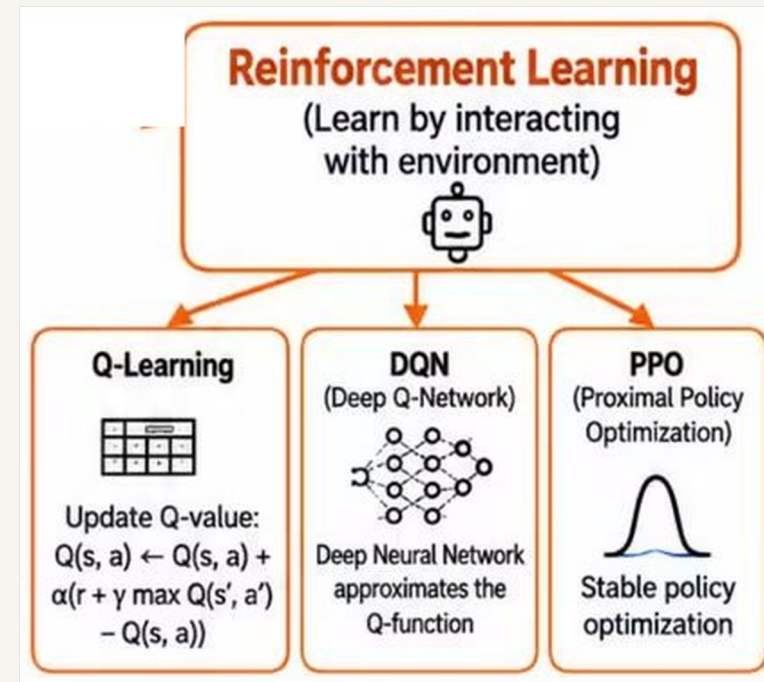
Reinforcement learning moved from values to policies

Reinforcement learning (learn by interacting) — lower-right panel of the original

Q-LEARNING → DQN → PPO

Tables to networks

- Q-learning — Q table
- DQN — Q network
- PPO — the policy itself



Only rewards, no labels — balancing exploration and exploitation is key

Three algorithms, three uses of the Bellman equation

Discount, learning rate, target network and policy ratio are the key parts

Bellman optimality equation

$$Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right]$$

Q-learning · TD update

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

DQN · loss against a target network

$$\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} Q_{\theta^-}(s', a') - Q_{\theta}(s, a) \right)^2$$

PPO · clipping limits policy change

$$L^{\text{CLIP}} = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

PPO keeps each policy step small, which stabilises learning

Choosing an algorithm starts with the data and the goal

The original's point — know when and why, not every name

Ask first

Family

Start with

Is y a number?

Regression

Linear · Ridge → GBM

Is y a class?

Classification

Logistic → XGBoost · LightGBM

Group without labels?

Clustering

K-Means → DBSCAN · GMM

Too many features?

Reduction

PCA → UMAP · Autoencoder

Sequential decisions?

Reinforcement

Q-Learning → DQN · PPO

Start with a simple baseline, then add complexity only if it beats it

**The learning signal sets the family,
the family sets the objective,
and the objective sets the algorithm.**

**Start simple, use real data,
and compare as you scale up**

※ Figures: excerpts from Pooja Pawar, "Machine Learning Algorithm Family Tree" infographic